

Adapter Architecture

Architecture of my adapter and how it integrates with the QWEN 2.5 3B model (or other base model)

My adapter architecture is based on the Low-Rank Adaptation (LoRA) method. Instead of training all the parameters of the base model, LoRA introduces small, trainable adapter matrices into the model's key layers. These adapters are designed to perform papermeters fine-tuning and capture the weight updates needed for the text summarization task with lower computational cost and memory efficiency.

The core of the LoRA adapter is a low-rank decomposition of the weight update matrix (ΔW). For any pre-trained weight matrix W_0 , the update ΔW is approximated as the product of two much smaller matrices, A and B.

$$\Delta W = BA$$

- Matrix A is a down-projection matrix with a shape of (r, din) , where r is the rank dimension.
- Matrix B is an up-projection matrix with a shape of (dout, r) .

Rank (r): controls the number of trainable parameters in the adapter. It creates a bottleneck, projecting the input from a high-dimensional space to a much smaller space and then back up. The number of trainable parameters is significantly reduced from $\text{din} \times \text{dout}$ to $r \times (\text{din} + \text{dout})$

The LoRA adapters will be integrated into the QWEN 2.5 3B model's **attention** and **feed-forward** layers.

The pre-trained weights of the base model are frozen and remain unchanged throughout the fine-tuning process. The trainable adapters are added directly to these frozen weights.

The integration process works as follows:

1. Placement: The adapters are strategically placed parallel to the **linear layers of the attention mechanism**, specifically within the **query** and **value** projection matrices. They can also be added to the feed-forward network layers.
2. Forward Pass: During a forward pass, the input is multiplied by the original frozen weight matrix (W_0) and by the product of the two small adapter

matrices (BA). The results are then summed. This can be expressed as:
 $y = W_0x + BAx$

3. Training: The training process only updates the weights of the two small adapter matrices, A and B. This is a major advantage because it drastically reduces the number of trainable parameters, leading to much faster fine-tuning and lower memory requirements compared to a full fine-tuning approach.
4. Inference: For inference, the two matrices A and B can be merged into the original weight matrix (W_0) by calculating the full update matrix (BA) and adding it to W_0 . The combined model, $W' = W_0 + BA$, can be used for inference, without additional latency.

Key components of the adapter and how they interact with each other

Base Model Qwen/Qwen2.5-3B

```
AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-3B", **kwargs)
```

This base model loads the weights of Qwen/Qwen2.5-3B model.

It is the backbone and its weights are not updated during parameter finetuning. This is represented by `param.requires_grad = False`.

LoRA LoraConfig(rank, lora_alpha, target_modules, drop_out, **kwargs)

Parameters:

- `r = 64`: Low-rank dimension (controls adapter capacity).
- `lora_alpha = 128`: Scaling factor to amplify updates.
- `target_modules = ["q_proj", "v_proj"]`: LoRA adapters are injected only into the attention layers' query and value projections.
- `lora_dropout = 0.05`: Dropout applied to LoRA layers for regularization.
- `bias = "none"`: No bias parameters are trained.
- `task_type = "CAUSAL_LM"`: causal language modeling.

This configuration decides the position and how the LoRA adapters will be attached to the frozen base model.

LoRA Adapters get_peft_model(base_model, lora_config)

This method wraps the base model with LoRA modules.

For each q_proj and v_proj in the attention blocks, a LoRA low-rank decomposition is added:

- Two trainable matrices (A and B) of rank r are injected alongside frozen weights.
- During training, updates flow through these small matrices instead of the full weight matrices.
- The forward pass combines frozen base weights with LoRA's updates

Clarify the reasons and advantages of this architecture.

This adapter architecture allows the QWEN 2.5 3B model to adapt to text summarisation tasks to **significantly reduce the computational cost and memory requirements** of fine-tuning large pre-trained models. The primary advantage is **parameter efficiency**.

Advantages

Parameter Efficiency: Instead of training all parameters in a model (e.g., QWEN 2.5B, which has 2.5 billion parameters), the LoRA architecture only trains a small fraction of them. This is because for a rank of 64, training is only done on a few million parameters.

Reduced Memory Usage: With fewer trainable parameters and no need to store full optimizer states for the entire model, the memory required for fine-tuning is substantially lower.

Faster Fine-Tuning: Because the backpropagation pass only needs to compute gradients for a small number of parameters, the training process is much faster.

No Inference Latency: When it comes time for inference, the LoRA adapters can be merged directly into the base model's weights. The final merged model is a single, fine-tuned model that performs inference without any added computational overhead or latency from the adapters.

Diversify Task: we can train multiple adapters different downstream tasks and swap in and out for the same base model. This allows us to have a single large model and easily fine-tune it for a variety of tasks without storing a separate full model for each.

Input and Output Formats

Input format for the summarization adapter

The input must be in the form of **tensors**. These tensors are:

- **Token IDs**: Text data is converted into numerical sequences of token IDs using a **tokenizer**. For example [101, 146, 735, 987, 102].
- **Attention Mask**: A binary tensor that tells the model which tokens to pay attention to and which to ignore. It is used to mask out padding tokens.

Scripts for data format conversion from/to plain text.

Link: [Data format conversion](#)

Output format for the summarization adapter.

Prediction post-processing steps:

1. Token Generation: The adapter and base model predict the most likely next token in a sequence, one at a time. The output is a tensor of logits, a sequence of token IDs
2. Decoding: the output tensor is then passed to the tokenizer's decode method. This method converts the numerical IDs back into a human-readable text string.
3. Final Output: The result will contain both the article and summary. This prediction text is then stripped to only keep the summary portion, which is then used to compare with the ground truth highlight/summary.

Final Output format: plain text.

For example:

Token generated:

```
tensor([ 9190, 5612, 551, 279, 2701, 1467, 1447, 3025, 9745, 8,
        5050, 6910, 369, 20322, 315, 279, 9742, 5780, 311, 30423,
        5016, 9400, 13, 576, 4741, 17121, 8953, 37158, 40970, 5926,
        13895, 1007, 264, 4013, 315, 4937, 45518, 323, 296, 12817,
        1495, 264, 1874, 315, 6271, 64585, 879, 1865, 279, 16523,
        315, 68715, 73160, 323, 26583, 18376, 6026, 6042,...])
```

This tensor then goes through decoding and the output has the same format as the input.

Output from decoding:

Summarize the following article.

(CNN)There was a street named after Chuck Norris, but they had to change the name because nobody crosses Chuck Norris and lives. Chuck Norris counted to infinity. Twice. Death once had a near-Chuck Norris experience. Chuck Norris is celebrating his 75th birthday -- but the calendar is only allowed to turn 39. That last one is true (well, the first part, anyway). The actor, martial-arts star and world's favorite tough-guy joke subject was born March 10, 1940, which makes him 75 today. Or perhaps he IS 39. Because maybe YOU can't beat time, but Chuck Norris can beat anything. Happy birthday!

Summary:

Tuesday is Chuck Norris' 75th birthday .

The actor and martial arts master is now known as subject of tough-guy one-liners.

Final output:

Tuesday is Chuck Norris' 75th birthday .

The actor and martial arts master is now known as subject of tough-guy one-liners.

Evaluation Metrics

Evaluation metrics to assess the quality of the summaries generated by the adapter, and compare with the base model.

For daily development and hyperparameter tuning

- **ROUGE:** Since this is a text summarization task, it is important to minimize false negatives. The predicted summary should capture as much relevant content from the source text as possible. ROUGE is suitable for this purpose, as it measures the degree of overlap between the generated summary and the reference summary.
- **BLEU** can also be used to assess the quality of the summaries by identifying text in the predictions that does not align with the reference. However, ROUGE is generally preferred for summarization because generating longer summaries is acceptable as long as they capture the key content of the source text.

- **BERTScore** evaluates semantic similarity between the predicted and reference summaries using contextual word embeddings. Unlike ROUGE or BLEU, it can account for synonyms and paraphrases, making it particularly useful for assessing meaning preservation and encouraging more diverse model outputs.

Qualitative metrics

Human Evaluation: Let human marks the predicted outputs, with the following method:

- Provide a Q&A sheet for the human evaluators with specific evaluation rubrics and ranking points for each rubric.
- Calculate the sum for these rubrics and aim to improve them

How to use these metrics to fine-tune and improve the performance of the adapter.

Metrics Calculation

- Compute mean ROUGE-1, ROUGE-2, and ROUGE-L (r1, r2, rL), BLEU and BERT scores on evaluation set.

Model Fine-Tuning

- Adjust **rank** and **lora_alpha**: Increasing rank (in range 12 - 68 for this dataset) and alpha (= rank*2 or rank*3) may help improve ROUGE and BLEU scores.
- Adjust **learning rate**: Decreasing the learning rate may improve ROUGE scores.
- Adjust learning rate: 3e-4, then if the model loss is relatively high, decreases to 1e-4, etc to improve ROUGE and BLEU scores.
- Varies lora_dropout ~ 0.5. Increase drop-out when the model starts to overfit. Otherwise, decrease when it underfits.
- Evaluation strategy="steps" or "epoch"

Loss Monitoring

- Track **training loss vs. evaluation loss** every ~50 steps to diagnose:
 - **Overfitting:** training loss decreases, while evaluation loss increases or stagnates.

- **Underfitting:** training loss remains high (e.g., > 3.0)

Performance Tracking

- Record the progression of each fine-tuned model on ROUGE, BLEU and BERT scores. (daily fine-tuning)
- Evaluate the generated summary texts qualitatively (human evaluation) in addition to quantitative scores.

Dataset

Dataset that I will use to train and evaluate my summarization adapter.

1. [abisee/cnn_dailymail](#) from HuggingFace (CNN_DailyMail)
2. [EdinburghNLP/xsum](#) from HF (XSum): online articles from BBC News

Characteristics of the dataset (e.g., size, domain, type of documents).

Dataset name	Average Length	Domain	Summary Length	Summarization Style	Type of documents
CNN_DailyMail	784.24	News	Multi-sentence, multi-paragraph	Mix of extractive and abstractive	text
XSum	410.44	News	Single-sentence	Highly abstractive	text

3. Create my own dataset with human written summaries - on multiple domains (books, short stories, movie reviews, social media posts, meeting minutes, etc...)

Implementation Plan

Step-by-step plan for implementing my adapter, including any preprocessing steps, training procedures, and post-processing techniques.

1. **Preprocessing** - from the dataset, make prompts with the articles as the input ("text") and highlights as the summary. The prompt will have this format:

Summarise the following text:

{Article}

Summary:

{Summary}

For example:

Summarise the following text:

In a groundbreaking study, researchers at the University of California have discovered a new method to significantly enhance the efficiency of solar panels. The method involves the use of a novel material that can absorb a broader spectrum of light, leading to a 20% increase in energy conversion rates. This breakthrough could revolutionize the renewable energy sector and pave the way for more sustainable and cost-effective solutions.

Summary:

Researchers at UC discover a new material to enhance solar panel efficiency by 20%, potentially revolutionizing the renewable energy sector.

- The prompt will be stored in the *input_texts* column.
- A new column called *labels* will be created, which contains the same text as the *input_texts* column.
- Next, both the *input_texts* and *labels* will be tokenized using the AutoTokenizer. During tokenization, the texts will be converted into token IDs along with their corresponding attention masks. Finally, the tokenized outputs will be formatted into PyTorch tensors to be used for model training.

2. Training procedures:

The model will be trained using the tokenized input texts, tokenized labels,, and their corresponding attention masks. During training, the model computes the loss based on the difference between the predicted outputs and the labels.

Every 50 steps, the model will be evaluated on the validation set to calculate the evaluation loss. To prevent overfitting, an **early stopping** will be implemented: if signs of overfitting appear (for example, training loss continues to decrease while evaluation loss increases or stagnates), the training process will be stopped..

3. Post-Processing techniques

Each evaluation input will be tokenized into input IDs, labels, and attention masks before being fed into the model.

The model will produce tokenized predictions, which is decoded back into natural language summaries. Only the summary portion of the decoded prediction will be extracted.

These predicted summaries will be compared against the reference highlights using evaluation metrics: **ROUGE**, **BLEU**, and **BERTScore**.

Discuss any challenges I anticipate and how I plan to address them.

- Since this model is a causal model, the prediction might not contain a summary.
To address this, I will increase rank and alpha: increase the model's learning capacity to learn to summarize text based on the input. However, one disadvantage is that a high rank leads to a higher risk of overfitting, especially on small datasets. It also increases memory usage and training time. To address this, I will use early stopping, dropout regularization, and possibly experiment with smaller learning rates. I will also experiment with dropout regularization, and possibly experiment with smaller learning rates.
- Summary might be longer than the original text/article (might include extra information from the input, or might repeat text/gibberish in the summary).
- The summary might copy directly the text from the summary and not summarising it.
- Underfitting: If the model shows consistently high training loss, I will adjust hyperparameters such as rank, lora_alpha, and learning rate to improve model capacity and optimization.

- Evaluation Metric Limitations: While ROUGE and BLEU capture lexical overlap, they may not fully reflect semantic quality. To mitigate this, I will include BERTScore and conduct a qualitative evaluation of sample summaries.

Source Code and Model

- **Source code:** https://github.com/thitran3/summarization_adapter
- **Trained adaptor model**
https://github.com/thitran3/summarization_adapter/tree/main/peft-text-summary

How to inference and test (with [test.py](#))

Inference steps:

1. Fine-tuned model: The adapter is attached back to the base model with method `PeftModel.from_pretrained(base_model, MODEL_PATH)`.
1. Input data preparation: New, raw input text data is pre-processed with `tokenizer()` to the same format as the model was trained on.
2. Model execution: The prepared data is fed into the fine-tuned model:, which performs calculations based on its learned parameters (`ft_model.generate()`).
3. Prediction generation: The model outputs a result, which is then decoded by `tokenizer.decode()` to output a text. This text is then split by `""Summary:\n"` to output only the summary text.
4. Output delivery: The output is printed out as text format.

FINE-TUNING ANALYSIS

ADAPTER PARAMETERS

	Rank	Alpha	Lora_drop_out
Base model			
Peft_1.0	4	8	1
Peft_2.0	6	8	1
Peft_3.0	6	12	0.05
Peft_4.0	12	24	0.05
Peft_5.0	24	48	0.05

TRAINING PARAMETERS

	Learning rate	Eval_strategy	Eval_steps
Base model			
Peft_1.0	3e-5	steps	20
Peft_2.0	3e-5	steps	20
Peft_3.0	3e-5	steps	20
Peft_4.0	1e-4	steps	50
Peft_5.0	1e-4	steps	50

TRAINING RESULTS

TRAINING LOSS

For each iteration of model training, I monitor the training loss to detect overfitting/underfitting. To prevent overfitting, I have implemented early stopping to stop the training process when the model starts to overfit.

A decrease in training loss at each training iteration also underfit less. I have seen a drop in training loss from peft_1.0 to peft_3.0. From peft_3.0 to peft_5.0, the decrease in training loss is not so significant.

Step	Training Loss	Validation Loss
50	No log	2.034489
100	2.105200	2.001315
150	2.105200	1.999090
200	2.016600	2.001047
250	2.016600	1.999554
300	2.005300	2.003772
350	2.005300	2.005094
400	2.000200	2.006617
450	2.000200	2.008960
500	1.978000	2.009754
550	1.978000	2.013396
600	1.966100	2.016763
650	1.966100	2.016016
700	1.989800	2.011021
750	1.989800	2.010492

ROUGE SCORES

Model	Mean ROUGE-1	Mean ROUGE-2	Mean ROUGE-L
Base	0.229943	0.223441	0.228464
peft_1.0	0.340242	0.32561	0.335208
peft_2.0	0.330166	0.319879	0.327651
peft_3.0	0.344314	0.330003	0.339659
peft_4.0	0.34652	0.33261	0.342025
peft_5.0	0.340983	0.326722	0.336727

SUMMARY COUNT

Model	Missing summary count	Missing summary percentage
Base model	122/200	61.00%
peft_1.0	122/200	61.00%
peft_2.0	116/200	57.99%
peft_3.0	114/200	56.99%
peft_4.0	116/200	57.99%
peft_5.0	114/220	56.99%

Base model

- Incomplete (the last line cuts at “The ferry” in example 1)
- Repetitive (“The ferry was carrying 212 people” was repeated, many times in example 1)

[illegible]

ground_truth	prediction
Erich Nowusch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other .	Erich Nowusch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other . the shooting happened after a road rage incident . Nowusch's attorney says his client will argue self-defense . The driver of the silver Audi is still at large Nowusch was arrested five days after the shooting . The shooting happened after a road rage incident . The driver of the silver Audi is still at large . Nowusch was arrested five days after the shooting . The shooting happened after a road rage incident . The driver of the silver Audi is still at large . Nowusch was arrested five days after the shooting . The shooting happened after a road rage incident . The driver of the silver Audi is still at large . News

ground_truth	prediction
The group is made up of one man, four women, 11 children . They were stopped just north of Aleppo, Syria .	The group is made up of one man, four women, 11 children . They were stopped just north of Aleppo, Syria . main objective is to cross the border into Syria . The Indonesian government is sending a security team to Turkey to investigate the group's objectives and plans and to increase cooperation with Turkish security officials . This group of 16 Indonesians is different from another 16 Indonesians who reportedly deserted their tour group recently in Turkey and are feared to have crossed into Syria . The Indonesian government is sending a security team to Turkey to investigate the group's objectives and plans and to increase cooperation with Turkish security officials . This group of 16 Indonesians is different from another 16 Indonesians who reportedly deserted their tour group recently in Turkey and are feared

Improvements

- ROUGE scores increase by around 0.1 compared to the base model.
- Fine-tune adapter on base model shows improvement in summary quality compared to base model.
- There is no more repetition in predictions.

- However some summaries are still incomplete.
- Number of missing summary still does not improve compared to base model (122/200 missing summaries in predictions)

ground_truth	prediction
Thousands march in a protest against terrorism in Tunisia's capital . Demonstrators hold signs that say "We are not afraid" and "Je suis Bardo" Tunisia's Prime Minister says a suspect in the Bardo museum attack was killed in a raid .	Thousands march in a protest against terrorism in Tunisia's capital . Demonstrators hold signs that say "We are not afraid" and "Je suis Bardo" Tunisia's Prime Minister says a suspect in the Bardo museum attack was killed in a raid . the CNN Wire
Erich Nowisch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other .	Erich Nowisch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other . the Meyers family

Peft_2.0

Improvements

- There is a 3% drop in the number of missing summaries in predictions compared to peft_1.0.

Limitations

- Some summaries are still incomplete
- There is a slight drop in ROUGE Scores.

ground_truth	prediction
Erich Nowisch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other .	Erich Nowisch will face three charges, including first-degree murder . He is accused of killing Tammy Meyers in front of her home . The two lived withing walking distance of each other . in front of her home

Conclusion

- Increase in rank from rank=4 to rank=6 increases the model ability to generate correct predictions.

Peft_3.0

Improvements

- There is a 1% drop in the number of missing summaries in predictions compared to peft_2.0.
- A slight increase in ROUGE Scores.

Limitations

- Some summaries are still incomplete

Conclusion

- An increase in alpha from alpha=8 to alpha=12 also contributes to the model's ability to generate correct predictions.

- A decrease in lora_dropout from 1.0 to 0.5 also contributes to the model's ability to generate correct predictions.

Peft_4.0

Improvements

- There is an increase in ROUGE Scores compared to peft_3.0.

Limitations

- There is a slight increase in the number of missing summaries in predictions.

Conclusion

- Increase of rank from rank=6 to rank=12 and alpha from alpha=12 to alpha=24 contributes to summaries generated being closer to the ground_truth summaries.
- Decrease of learning rate also contributes to improvement in the model ability to generate summaries.

Peft 5.0

Improvements

- A slight drop in the number of missing summaries compared to peft_4.0.

Limitations

- There is a slight drop in ROUGE Scores compared to peft_4.0
- Rouge scores did not improve compared to peft_2.0 and peft_3.0.

Conclusion

- Higher rank of ≥ 24 and $\alpha \geq 48$ does not improve the model ability to generate correct predictions.

FINAL CONCLUSION

- High rank and alpha does help the model to be able to generate summaries better. However, due to the limitation of the training set (small size), a rank of 24 and above does not make a significant role in helping the model learn better.
- An increase in learning rate and decrease in lora_drop_out also helps to improve the model.